Tutorial 7

Support Vector Machines and Least-Squares SVM

Department of Computer Science
City University of Hong Kong

Outline

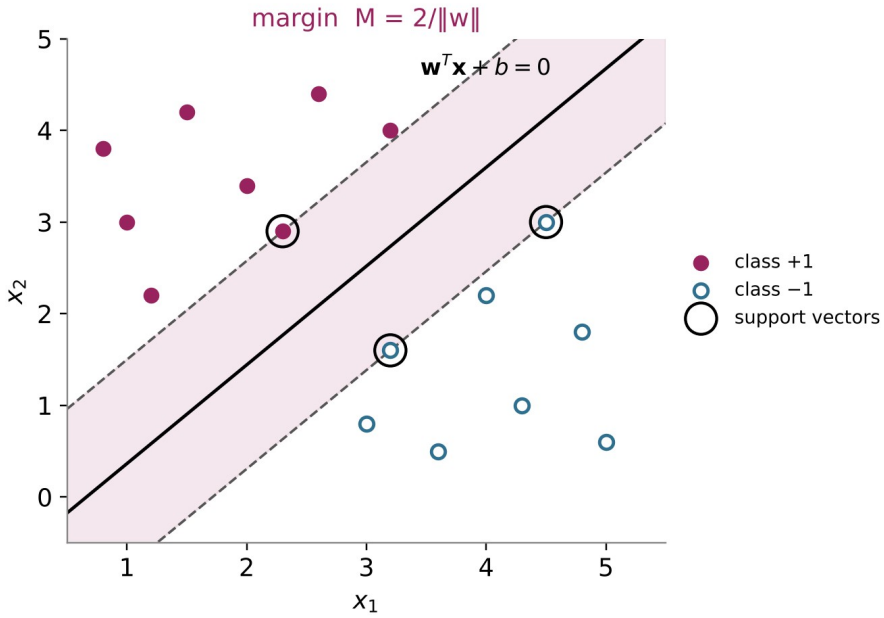
1. Linear SVM: Maximal Margin
2. Dual Problem and Support Vectors
3. Soft Margin
4. Kernel Trick and Nonlinear SVM
5. Support Vector Regression
6. Least-Squares SVM
7. SVM and LS-SVM in MATLAB
8. Quick Check

1. Linear SVM: Maximal Margin

A linear classifier:

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$

- g(x) > 0 → class +1, g(x) < 0 → class −1
- Many hyperplanes separate the data. Which one generalizes best?
- SVM picks the hyperplane with the **maximal margin**: the widest "safe zone" between the classes
- Points on the margin boundaries are the **support vectors**



1. Margin and the Primal Problem

Scale **w** and *b* so that the closest points satisfy $\mathbf{w}^T x + b = \pm 1$. Then the margin width is

$$M = \frac{2}{\|\mathbf{w}\|}$$

Maximizing $2/\|w\|$ is equivalent to minimizing $\frac{1}{2}\|w\|^2$:

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad i = 1, \ldots, n \end{aligned}$$

The constraint means: every point is on the correct side **and** outside the margin.

2. Lagrangian and Dual Problem

1. Add a multiplier $\alpha_i \geq 0$ for each constraint:

$$L_p = \tfrac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \big(y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big), \quad \alpha_i \geq 0$$

2. Set the derivatives to zero:

$$\frac{\partial L_p}{\partial \mathbf{w}} = 0 \;\Rightarrow\; \mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i, \qquad \frac{\partial L_p}{\partial b} = 0 \;\Rightarrow\; \sum_{i=1}^{n} \alpha_i y_i = 0$$

3. Substitute back → the **dual problem** (only α_i are unknown):

$$\max_{\boldsymbol{\alpha}} \quad \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^T \mathbf{x}_j$$

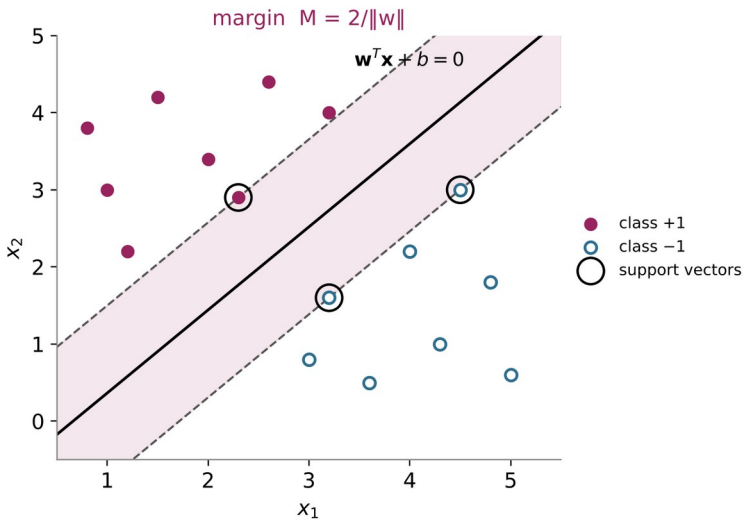
$$\text{s.t.} \quad \alpha_i \geq 0, \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

Key observation: the data appear only through dot products $x_i^T x_j$.

2. KKT Conditions and Support Vectors

One result from the KKT conditions:

$$\alpha_i \big(y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \big) = 0$$



Point outside the margin

→ $\alpha_i = 0$, no effect on w

Point on the margin

→ $\alpha_i > 0$: a **support vector**

So $\mathbf{w} = \Sigma\, \alpha_i\, y_i\, \mathbf{x}_i$ is determined **only by the support vectors**.

3. Soft Margin: Slack Variables and C

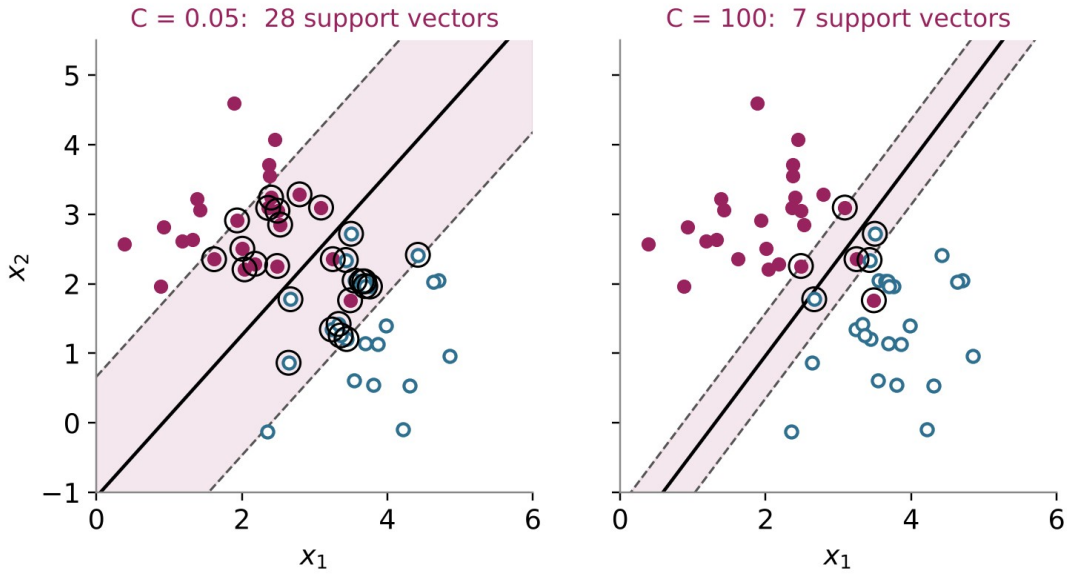
Noisy or overlapping data → allow violations with slack $\xi_i \geq 0$

$$\begin{aligned} \min \quad & \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

Dual problem

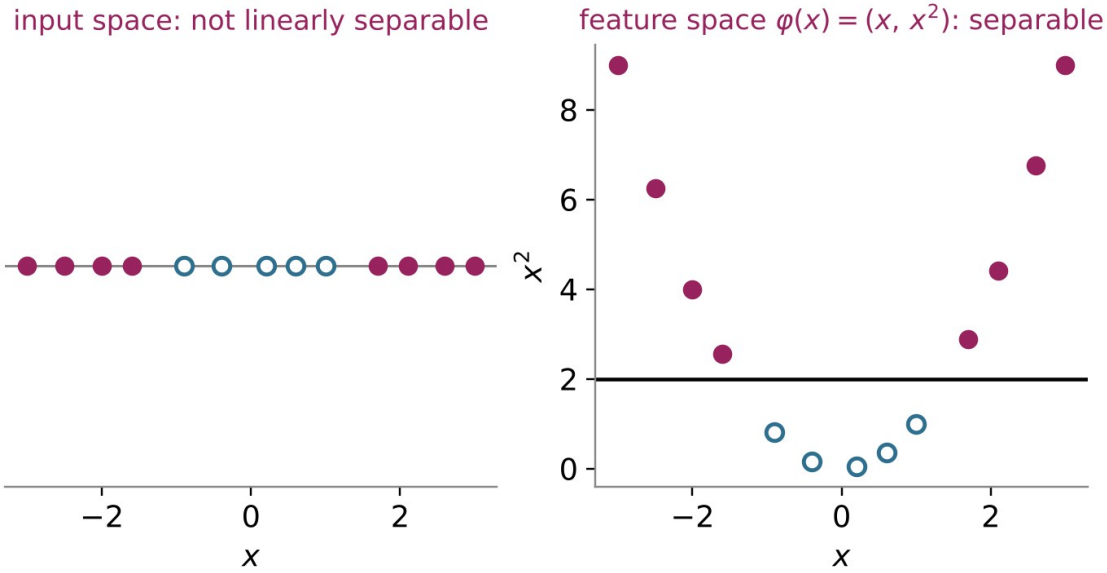
$$0 \leq \alpha_i \leq C$$

Same as before, plus an upper bound



4. The Kernel Trick

Idea: map **x → φ(x)** into a higher-dimensional feature space where the classes become linearly separable.



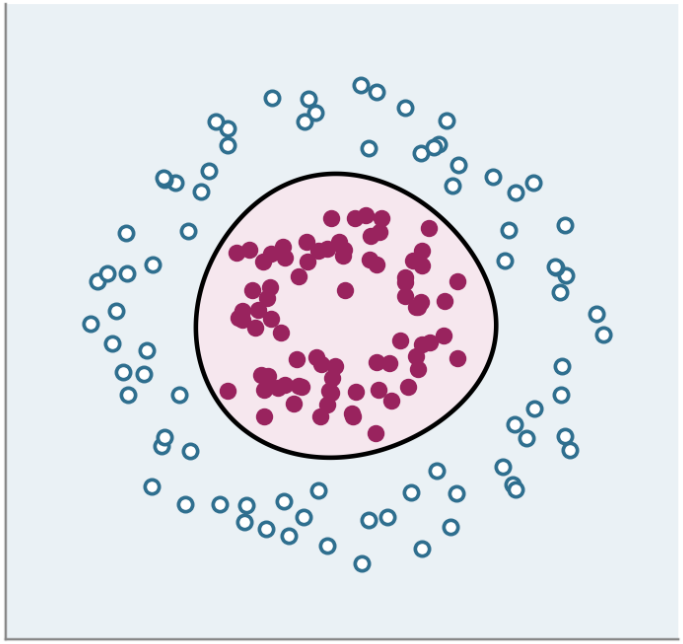
Training and testing only need dot products, so we use a **kernel** instead of computing φ:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$$

4. Common Kernel Functions

Linear	$\mathbf{x}_i^T \mathbf{x}_j$
Polynomial	$(1 + \mathbf{x}_i^T \mathbf{x}_j)^p$
Gaussian (RBF)	$\exp\left(- \frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2} \right)$
Sigmoid	$\tanh(\beta_0 \mathbf{x}_i^T \mathbf{x}_j + \beta_1)$

RBF-kernel SVM boundary



The RBF kernel is the usual default choice.

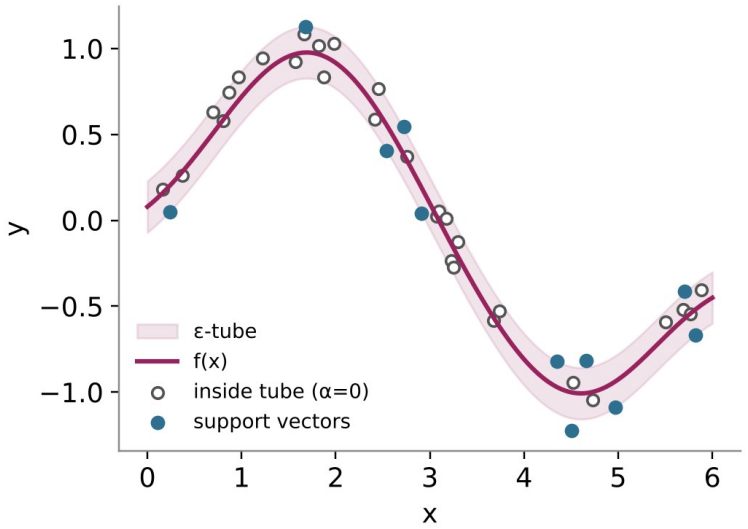
4. SVM Learning in Practice

1. **Choose a kernel**
   e.g. Gaussian (RBF) or polynomial
2. **Choose C**
   and kernel parameters, e.g. σ
3. **Solve the QP**
   many software packages available
4. **Build g(x)**
   from the support vectors

5. Support Vector Regression

SVM can also do **regression**: predict a number instead of a class.

Idea: fit f(x) with an **ε-tube** around it.



Point inside the tube

→ error ignored, $\alpha_i = 0$

Point outside the tube

→ penalized: a **support vector**

6. Least-Squares SVM (LS-SVM)

A simpler version of SVM, proposed by **Suykens and Vandewalle** (1999).

$$\min_{\mathbf{w},b,e} \quad \tfrac{1}{2}\mathbf{w}^T\mathbf{w} + \gamma\,\tfrac{1}{2}\sum_{i=1}^{N} e_i^2$$

$$\text{s.t.} \quad y_i = \mathbf{w}^T\varphi(\mathbf{x}_i) + b + e_i, \quad i = 1, \ldots, N$$

½ $\mathbf{w}^T\mathbf{w}$

Keeps the model simple (same as SVM)

$\gamma \cdot$ ½ $\Sigma\, e_i^2$

Squared error of every point. **γ (gamma)** = how much errors matter, like C

Constraint

Each target = model prediction + error e_i

Squared error + equality constraints → train by solving **one linear system** (no QP).

6. LS-SVM: Training = One Linear System

Using the same Lagrangian method, training becomes **one linear system**:

$$\left[\begin{array}{c|c} \boldsymbol{\Omega} + \gamma^{-1}\mathbf{I} & \mathbf{1} \\ \hline \mathbf{1}^T & 0 \end{array}\right] \left[\begin{array}{c} \boldsymbol{\alpha} \\ \hline b \end{array}\right] = \left[\begin{array}{c} \mathbf{y} \\ \hline 0 \end{array}\right], \qquad \Omega_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$$

Solve it once → get **α** and **b** (in MATLAB: one backslash **A \ y**)

Final model:

$$y(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i \, K(\mathbf{x}_i, \mathbf{x}) + b$$

6. SVM vs. LS-SVM

	SVM	LS-SVM
Loss	Hinge (classification) / ε-insensitive (regression)	Squared error e_i^2
Constraints	Inequality	Equality
Training	Quadratic programming	Solve one linear system
Sparseness	Sparse: only support vectors have $\alpha_i \neq 0$	Not sparse: $\alpha_i = \gamma e_i$, usually $\neq 0$
Hyper-parameters	C + kernel parameters (e.g. σ)	γ + kernel parameters (e.g. σ^2)

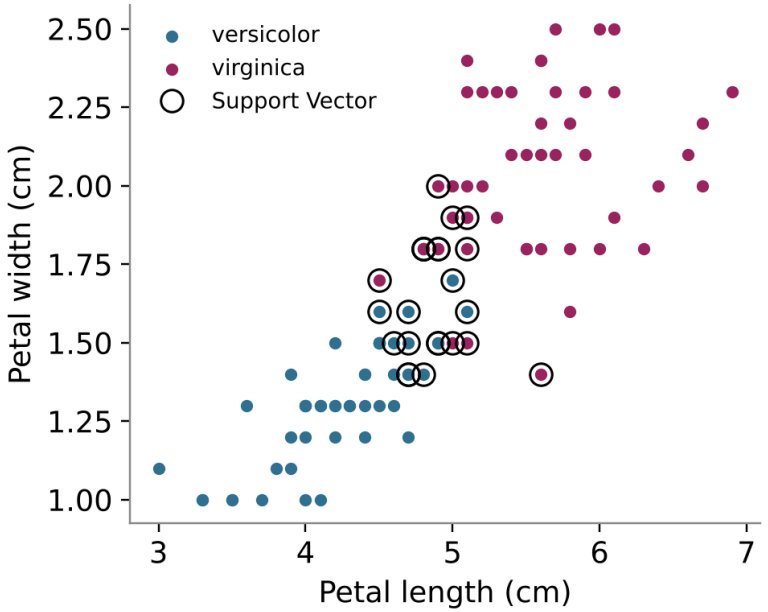
Both share the same final form: **$f(x) = \Sigma\, \alpha_i\, K(x_i, x) + b$** (SVM classification uses $\alpha_i y_i$)

7. MATLAB: fitcsvm (Binary Classification)

```
% example_1.m
load fisheriris
inds = ~strcmp(species,'setosa');
X = meas(inds,3:4);    % petal length, width
y = species(inds);

SVMModel = fitcsvm(X,y)   % linear, C = 1
classOrder = SVMModel.ClassNames

sv = SVMModel.SupportVectors;
figure
gscatter(X(:,1),X(:,2),y)
hold on
plot(sv(:,1),sv(:,2),'ko','MarkerSize',10)
legend('versicolor','virginica',...
       'Support Vector')
hold off
```



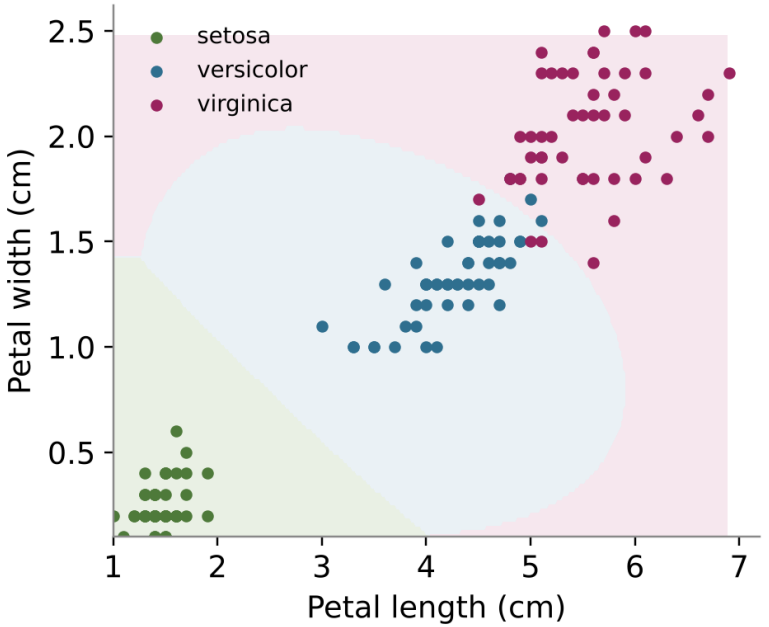
Expected plot (illustration)

- **`'KernelFunction'`**: 'linear' (default), 'rbf', 'polynomial'
- **`'BoxConstraint'`** is C; **`'Standardize', true`** scales the inputs

7. MATLAB: RBF Kernel and Multiclass

```
% example_2.m: one-vs-all, RBF kernel
for j = 1:numel(classes)
  indx = strcmp(Y,classes(j));
  SVMModels{j} = fitcsvm(X,indx, ...
     'ClassNames',[false true], ...
     'Standardize',true, ...
     'KernelFunction','rbf', ...
     'BoxConstraint',1);
end
% predict: pick the class with max score
```

```
% example_3.m: automatic multiclass
t = templateSVM('Standardize',true);
Mdl = fitcecoc(meas,species,'Learners',t);
CVMdl = crossval(Mdl);        % 10-fold CV
genError = kfoldLoss(CVMdl)
```



Classification regions (illustration)

`fitcecoc` combines binary SVMs for multiclass problems; **`kfoldLoss`** estimates generalization error.

7. MATLAB: fitrsvm (Support Vector Regression)

```
% example_4.m
load carsmall
rng 'default'
X = [Horsepower,Weight];
Y = MPG;

Mdl = fitrsvm(X,Y);
Mdl.ConvergenceInfo.Converged      % 0

MdlStd = fitrsvm(X,Y,'Standardize',true);
MdlStd.ConvergenceInfo.Converged   % 1

lStd = resubLoss(MdlStd)   % training MSE
```

Without standardizing

Converged = 0: the optimizer did not converge

With 'Standardize', true

Converged = 1: inputs on the same scale

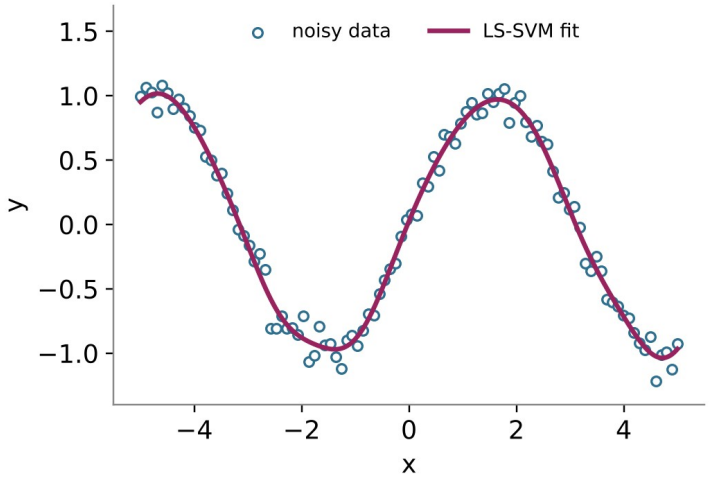
- **Useful options:** **`'Epsilon'`** (tube width ε), **`'BoxConstraint'`** (C), **`'KernelFunction'`**
- **`predict(Mdl, Xnew)`** gives predictions for new data

7. MATLAB: LS-SVM from Scratch

```
% example_5_lssvm.m
rng(1);
x = linspace(-5,5,100)';
y = sin(x) + 0.1*randn(size(x));
gam = 10;  sig2 = 0.5;   % gamma, sigma^2

N = numel(x);
Omega = exp(-(x - x').^2/(2*sig2));
A = [Omega + eye(N)/gam, ones(N,1);
     ones(1,N),          0       ];
sol = A \ [y; 0];          % one linear solve
alpha = sol(1:N);  b = sol(end);

xt = linspace(-5,5,400)';
Kt = exp(-(xt - x').^2/(2*sig2));
yt = Kt*alpha + b;          % LS-SVM model
plot(x,y,'o', xt,yt,'r-','LineWidth',2)
```



- All 100 α_i are non-zero → **not sparse**
- Try other γ and σ^2 to see under- and over-fitting

Toolbox option: LS-SVMlab (KU Leuven) provides **`trainlssvm`** and **`simlssvm`**

8. Quick Check

1. After training an SVM, which training points determine w and b?

2. In the soft-margin SVM, what happens to the margin as $C \to \infty$? And for small C?

3. For 2-D inputs, show that $(1 + x^T z)^2$ is a valid kernel. What is the feature map $\varphi(x)$?

4. Why is the LS-SVM solution not sparse, unlike the standard SVM?

Thank You !

9. Reference

[1] CS5486 lecture notes: Support Vector Machine and Least-squares SVM (slides 138–174)

[2] J. A. K. Suykens and J. Vandewalle, “Least squares support vector machine classifiers,” Neural Processing Letters, vol. 9, no. 3, pp. 293–300, 1999

[3] MATLAB fitcsvm: https://www.mathworks.com/help/stats/fitcsvm.html

[4] MATLAB fitrsvm: https://www.mathworks.com/help/stats/fitrsvm.html

[5] MATLAB SVM classification: https://www.mathworks.com/help/stats/support-vector-machine-classification.html

[6] LS-SVMlab toolbox: https://www.esat.kuleuven.be/sista/lssvmlab/